

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 12.

**Тема: Отладка программы с использованием
специализированных средств отладки**

Выполнила: Потанина Н.А.,

Студентка группы: 24-ИС.

2026г.

Цель работы:

изучить способы отладки программы с использованием языка программирования Python.

Вариант: №2

1. ПЕРВИЧНЫЙ АНАЛИЗ (Этап 1)

```
import math
import tracemalloc
import sys
import logging

# Глобальный список для логов — потенциальная утечка памяти
LOG_HISTORY = []

class TreeNode:
    """Узел дерева для рекурсивного обхода"""
    def __init__(self, value, children=None):
        self.value = value
        self.children = children if children is not None else []

def build_tree(depth, width):
    """Построение дерева заданной глубины и ширины"""
    if depth == 0:
        return None

    root = TreeNode(1)
    current_level = [root]

    for level in range(1, depth):
        next_level = []
        for node in current_level:
            for i in range(width):
                child = TreeNode(node.value + i + 1)
                node.children.append(child)
                next_level.append(child)
            current_level = next_level

    return root

def calculate_average(data_row):
    """Расчет среднего значения с логической ошибкой"""
    total = 0
    count = 0

    # ОШИБКА 1: Выход за границу списка
    for i in range(len(data_row) + 1): # +1 вызывает IndexError
        total += data_row[i]
        count += 1

    # ОШИБКА 2: Деление на ноль
    return total / count

def process_node(node, depth=0):
    """Рекурсивный обход дерева с ошибкой"""
    # ОШИБКА 3: Неправильное условие выхода
    if depth > 5: # Слишком маленький лимит
        return []
```

```

# ОШИБКА 4: Накопление в глобальном списке (утечка памяти)
LOG_HISTORY.append(f"Processing node {node.value} at depth {depth}")

result = [node.value]

# ОШИБКА 5: Рекурсивный вызов без увеличения глубины
for child in node.children:
    result.extend(process_node(child, depth)) # depth не увеличивается!

return result

def process_data(data_matrix):
    """Основная функция обработки матрицы данных"""
    results = []

    for row in data_matrix:
        avg = calculate_average(row)
        results.append(avg)

    return results

def analyze_tree(tree):
    """Анализ дерева с дополнительной обработкой"""
    if tree is None:
        return []

    values = process_node(tree, 0)
    return sorted(values)

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    logger = logging.getLogger(__name__)

    tracemalloc.start()

    test_data = [
        [10, 20, 30, 40],
        [5, 15, 25], # Здесь будет IndexError
        [100, 200, 300, 400],
        [], # Здесь будет ZeroDivisionError
    ]

    print("=== Запуск обработки данных ===")
    try:
        results = process_data(test_data)
        print(f"Результаты: {results}")
    except Exception as e:
        logger.error(f"Ошибка: {e}")
        import traceback
        traceback.print_exc()

    print("\n=== Построение дерева ===")
    try:
        tree = build_tree(10, 3)
        analysis = analyze_tree(tree)
        print(f"Анализ дерева: {len(analysis)} элементов")
    
```

except Exception as e:

logger.error(f"Ошибка: {e}")

traceback.print_exc()

snapshot = tracemalloc.take_snapshot()

print("\n==== Топ-5 строк по потреблению памяти ====")

for stat in snapshot.statistics('lineno')[:5]:

print(stat)

print(f"\nРазмер LOG_HISTORY: {len(LOG_HISTORY)} записей")

```
= RESTART: Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original
.PY
=== Запуск обработки данных ===
ERROR:__main__:Ошибка: list index out of range
Traceback (most recent call last):
  File "Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py",
    line 97, in <module>
    results = process_data(test_data)
  File "Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py",
    line 69, in process_data
    avg = calculate_average(row)
  File "Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py",
    line 41, in calculate_average
    total += data_row[i]
            ~~~~~^^^
IndexError: list index out of range

=== Построение дерева ===
Анализ дерева: 29524 элементов

=== Топ-5 строк по потреблению памяти ===
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py:27: siz
e=2540 KiB, count=29523, average=88 B
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py:54: siz
e=2259 KiB, count=29525, average=78 B
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py:13: siz
e=1614 KiB, count=29504, average=56 B
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py:28: siz
e=308 KiB, count=9841, average=32 B
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_original.py:80: siz
e=231 KiB, count=1, average=231 KiB

Размер LOG_HISTORY: 29524 записей
```

Выявленные проблемы на Этапе 1:

1. **IndexError** - основная ошибка, вызывает падение программы
2. **ZeroDivisionError** - скрытая ошибка (будет на строке [])
3. **RecursionError** - ошибка в рекурсии (depth не увеличивается)
4. **Утечка памяти** - LOG_HISTORY накапливает 29524 записи

Обоснование:

1. **Запуск скрипта** - программа запущена (variant_2_original.py)
2. **Фиксация ошибки** - получен полный traceback с IndexError
3. **Определение типа ошибки** - выявлена первая ошибка: IndexError: list index out of range
4. **Анализ стека вызовов** - виден путь от точки входа до места ошибки
5. **Первичный анализ памяти** - выведена статистика tracemalloc

Этап 2: Локализация с помощью breakpoint()

```

"""
variant_2_stage2_interactive.py
Интерактивная версия для работы с pdb
"""

import math
import tracemalloc
import sys
import logging
import pdb

# ===== НАСТРОЙКА =====
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
LOG_HISTORY = []

# ===== КЛАССЫ =====
class TreeNode:
    def __init__(self, value, children=None):
        self.value = value
        self.children = children if children is not None else []

# ===== ФУНКЦИИ =====
def build_tree(depth, width):
    if depth == 0:
        return None

    root = TreeNode(1)
    current_level = [root]

    for level in range(1, depth):
        next_level = []
        for node in current_level:
            for i in range(width):
                child = TreeNode(node.value + i + 1)
                node.children.append(child)
                next_level.append(child)
            current_level = next_level

    return root

def calculate_average(data_row):
    """Функция с ошибками - здесь будет breakpoint"""
    total = 0
    count = 0

    # ===== BREAKPOINT =====
    breakpoint() # <-- ТОЧКА ОСТАНОВА
    # =====

    # ОШИБКА: выход за границы
    for i in range(len(data_row) + 1):
        total += data_row[i]
        count += 1

    # ОШИБКА: деление на ноль

```

```

return total / count

def process_node(node, depth=0):
    if depth > 5:
        return []

    LOG_HISTORY.append(f"Processing node {node.value} at depth {depth}")

    result = [node.value]

    # ОШИБКА: depth не увеличивается
    for child in node.children:
        result.extend(process_node(child, depth))

    return result

def process_data(data_matrix):
    results = []
    for row in data_matrix:
        avg = calculate_average(row)
        results.append(avg)
    return results

def analyze_tree(tree):
    if tree is None:
        return []
    values = process_node(tree, 0)
    return sorted(values)

# ===== ОСНОВНАЯ ПРОГРАММА =====
if __name__ == "__main__":
    print("="*60)
    print("ИНТЕРАКТИВНАЯ ОТЛАДКА С breakpoint()")
    print("="*60)
    print("\nИнструкция:")
    print("1. Программа остановится на breakpoint()")
    print("2. Используйте команды pdb:")
    print("   - p data_row - просмотр данных")
    print("   - p len(data_row) - просмотр длины")
    print("   - n - выполнить следующую строку")
    print("   - where - показать стек вызовов")
    print("   - c - продолжить выполнение")
    print("   - q - выйти")
    print("="*60)

    tracemalloc.start()

    test_data = [
        [10, 20, 30, 40],
        [5, 15, 25],      # Здесь IndexError
        [100, 200, 300, 400],
        [],                # Здесь ZeroDivisionError
    ]

    print("\nТестовые данные:")
    for i, row in enumerate(test_data, 1):
        print(f"Строка {i}: {row}")

```

```
print("\nНачинаем обработку...")
print("-"*60)
```

```
try:
    results = process_data(test_data)
    print(f"\nРезультаты: {results}")
except Exception as e:
    print(f"\nОшибка: {e}")
    import traceback
    traceback.print_exc()
```

```
# Дерево
print("\n" + "="*60)
print("Построение дерева...")
print("="*60)
```

```
try:
    tree = build_tree(8, 3)
    analysis = analyze_tree(tree)
    print(f"Элементов: {len(analysis)}")
    print(f"Первые 10: {analysis[:10]}")
except Exception as e:
    print(f"Ошибка: {e}")
```

```
# Статистика
snapshot = tracemalloc.take_snapshot()
print("\n" + "="*60)
print("Статистика памяти:")
print("="*60)
for stat in snapshot.statistics('lineno')[:5]:
    print(stat)
print(f"\nLOG HISTORY: {len(LOG HISTORY)} записей")
```

```
=====
ИНТЕРАКТИВНАЯ ОТЛАДКА С breakpoint()
=====
```

Инструкция:

1. Программа остановится на breakpoint()
2. Используйте команды pdb:
 - p data_row - просмотр данных
 - p len(data_row) - просмотр длины
 - n - выполнить следующую строку
 - where - показать стек вызовов
 - c - продолжить выполнение
 - q - выйти

Тестовые данные:

```
Строка 1: [10, 20, 30, 40]
Строка 2: [5, 15, 25]
Строка 3: [100, 200, 300, 400]
Строка 4: []
```

Начинаем обработку...

```
-----
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(49)calculate_average()
-> breakpoint()  # <-- ТОЧКА ОСТАНОВА
(Pdb) |
```

```
ИНТЕРАКТИВНАЯ ОТЛАДКА С breakpoint()
```

```
=====
```

Инструкция:

1. Программа остановится на breakpoint()
2. Используйте команды pdb:
 - p data_row - просмотр данных
 - p len(data_row) - просмотр длины
 - n - выполнить следующую строку
 - where - показать стек вызовов
 - c - продолжить выполнение
 - q - выйти

```
=====
```

Тестовые данные:

```
Строка 1: [10, 20, 30, 40]
Строка 2: [5, 15, 25]
Строка 3: [100, 200, 300, 400]
Строка 4: []
```

Начинаем обработку...

```
-----
```

```
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(49)calculate_average()
```

```
-> breakpoint() # <-- ТОЧКА ОСТАНОВА
```

```
(Pdb) p data_row
```

```
[10, 20, 30, 40]
```

```
(Pdb) p len(data_row)
```

```
4
```

```
(Pdb) p range(len(data_row) + 1)
```

```
range(0, 5)
```

```
(Pdb) n
```

```
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(53)calculate_average()
```

```
-> for i in range(len(data_row) + 1):
```

```
(Pdb) p i
```

```
4
```

```
(Pdb) n
```

```
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(54)calculate_average()
```

```
-----
```

```
(Pdb) p i
```

```
4
```

```
(Pdb) n
```

```
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(54)calculate_average()
```

```
-> total += data_row[i]
```

```
(Pdb) p i
```

```
0
```

```
(Pdb) n
```

```
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(55)calculate_average()
```

```
-> count += 1
```

```
(Pdb) p i
```

```
0
```

```
(Pdb) n
```

```
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(53)calculate_average()
```

```
-> for i in range(len(data_row) + 1):
```

```
(Pdb) p i
```

```
0
```

```
(Pdb) p data_row[3]
```

```
40
```

```
(Pdb) where
```

```
<string>(1)<module>()
```

```
c:\users\student237\appdata\local\programs\python\python313\lib\idlelib\run.py
(172)main()
```

```
-> ret = method(*args, **kwargs)
```

```
c:\users\student237\appdata\local\programs\python\python313\lib\idlelib\run.py
(590)runcode()
```

```
-> exec(code, self.locals)
```

```
c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(120)<module>()
```

```
-> results = process_data(test_data)
```

```
c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(77)process_data()
```

```
-> avg = calculate_average(row)
```

```
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(53)calculate_average()
```

```
-> for i in range(len(data_row) + 1):
```

```
(Pdb) up
```



```

File Edit View Options Window Help
-> avg = calculate_average(row)
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(53)calculate_average()
-> for i in range(len(data_row) + 1):
(Pdb) up
> c:\users\student237\appdata\local\programs\python\python313\variant_2_stage2_i
nteractive.py(77)process_data()
-> avg = calculate_average(row)
(Pdb) pp locals()
{'data_matrix': [[10, 20, 30, 40], [5, 15, 25], [100, 200, 300, 400], []],
 'results': [],
 'row': [10, 20, 30, 40]}
(Pdb) c

Ошибка: list index out of range
Traceback (most recent call last):
  File "C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_st
age2_interactive.py", line 120, in <module>
    results = process_data(test_data)
  File "C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_st
age2_interactive.py", line 77, in process_data
    avg = calculate_average(row)
  File "C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_st
age2_interactive.py", line 54, in calculate_average
    total += data_row[i]
    ~~~~~^
IndexError: list index out of range

=====
Построение дерева...
=====
Элементов: 3280
Первые 10: [1, 2, 3, 3, 4, 4, 4, 4, 5, 5]

=====
Статистика памяти:
=====
C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_stage2_int
eractive.py:36: size=285 KiB, count=3279, average=89 B
C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_stage2_int

```

```

=====
Построение дерева...
=====
Элементов: 3280
Первые 10: [1, 2, 3, 3, 4, 4, 4, 4, 5, 5]

=====
Статистика памяти:
=====
C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_stage2_int
eractive.py:36: size=285 KiB, count=3279, average=89 B
C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_stage2_int
eractive.py:64: size=253 KiB, count=3281, average=79 B
C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_stage2_int
eractive.py:22: size=179 KiB, count=3279, average=56 B
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\linecache.py:172
: size=73.5 KiB, count=834, average=90 B
C:/Users/Student237/AppData/Local/Programs/Python/Python313/variant_2_stage2_int
eractive.py:37: size=34.2 KiB, count=1093, average=32 B

LOG_HISTORY: 3280 записей

```

Основная проблемная переменная: i

ЭТАП 3: Условные точки останова

"""

variant_2_stage3_fixed.py

"""

import math

import tracemalloc

import sys

import logging

from collections import deque

===== НАСТРОЙКА =====

```

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
LOG_HISTORY = deque(maxlen=100)

# ===== КЛАССЫ =====
class TreeNode:
    def __init__(self, value, children=None):
        self.value = value
        self.children = children if children is not None else []

# ===== ФУНКЦИИ =====
def build_tree(depth, width):
    if depth == 0:
        return None

    root = TreeNode(1)
    current_level = [root]

    for level in range(1, depth):
        next_level = []
        for node in current_level:
            for i in range(width):
                child = TreeNode(node.value + i + 1)
                node.children.append(child)
                next_level.append(child)
            current_level = next_level

    return root

def calculate_average(data_row):
    """Расчет среднего - с УСЛОВНЫМ breakpoint"""
    total = 0
    count = 0

    # =====
    # УСЛОВНЫЙ BREAKPOINT - сработает ТОЛЬКО для len=3
    # =====
    if len(data_row) == 3:
        print(f"\n>>> УСЛОВНАЯ ОСТАНОВКА! data_row = {data_row} <<<")
        breakpoint() # Остановка только для [5, 15, 25]
    # =====

    # ВАЖНО: добавили защиту от пустого списка
    if not data_row:
        logger.warning(f"Пустая строка, возвращаем 0")
        return 0

    # ИСПРАВЛЯЕМ ОШИБКУ: убираем +1
    for i in range(len(data_row)): # <-- ИСПРАВЛЕНО!
        total += data_row[i]
        count += 1

    return total / count

def process_node(node, depth=0, max_depth=10):

```

```

if depth > max_depth:
    return []

LOG_HISTORY.append(f'Node {node.value} at depth {depth}')
result = [node.value]

# Исправляем ошибку рекурсии
for child in node.children:
    result.extend(process_node(child, depth + 1, max_depth)) # <-- ИСПРАВЛЕНО!

return result

def process_data(data_matrix):
    results = []

    for idx, row in enumerate(data_matrix):
        logger.info(f'Обработка строки {idx+1}: {row}')
        avg = calculate_average(row)
        results.append(avg)

    return results

def analyze_tree(tree):
    if tree is None:
        return []
    values = process_node(tree, 0, 10)
    try:
        return sorted(values)
    except TypeError:
        return sorted([v for v in values if isinstance(v, (int, float))])

# ===== ОСНОВНАЯ ПРОГРАММА =====
if __name__ == "__main__":
    tracemalloc.start()

    print("="*60)
    print("ЭТАП 3: УСЛОВНЫЕ ТОЧКИ ОСТАНОВА (ИСПРАВЛЕНО)")
    print("="*60)

    test_data = [
        [10, 20, 30, 40], # len=4 - пропускаем
        [5, 15, 25],      # len=3 - ОСТАНОВКА!
        [100, 200, 300, 400], # len=4 - пропускаем
        [],                # len=0 - пропускаем
    ]

    print("\nТестовые данные:")
    for i, row in enumerate(test_data, 1):
        print(f"Строка {i}: {row} (длина={len(row)})")

    print("\n" + "-"*60)
    print("УСЛОВИЕ ОСТАНОВКИ: len(data_row) == 3")
    print("Остановка ТОЛЬКО на строке [5, 15, 25]")
    print("-"*60)

    print("\nЗапуск обработки...")
    print("="*60)

```

```

try:
    results = process_data(test_data)
    print(f"\n✅ Результаты: {results}")
except Exception as e:
    print(f"\n❌ Ошибка: {e}")
    import traceback
    traceback.print_exc()

print("\n" + "="*60)
print("ПОСТРОЕНИЕ ДЕРЕВА:")
print("="*60)

try:
    tree = build_tree(8, 3)
    analysis = analyze_tree(tree)
    print(f"Элементов: {len(analysis)}")
    print(f"Первые 10: {analysis[:10]}")
except Exception as e:
    print(f"Ошибка: {e}")

# Статистика памяти
snapshot = tracemalloc.take_snapshot()
print("\n" + "="*60)
print("СТАТИСТИКА ПАМЯТИ:")
print("="*60)
for stat in snapshot.statistics('lineno')[:5]:
    print(stat)
print(f"\nLOG_HISTORY: {len(LOG_HISTORY)} записей")

print("\n" + "="*60)
print("ЭТАП 3 ЗАВЕРШЕН!")
print("="*60)

```

```
=====
ЭТАП 3: УСЛОВНЫЕ ТОЧКИ ОСТАНОВА (ИСПРАВЛЕНО)
=====

Тестовые данные:
  Строка 1: [10, 20, 30, 40] (длина=4)
  Строка 2: [5, 15, 25] (длина=3)
  Строка 3: [100, 200, 300, 400] (длина=4)
  Строка 4: [] (длина=0)

-----

УСЛОВИЕ ОСТАНОВКИ: len(data_row) == 3
Остановка ТОЛЬКО на строке [5, 15, 25]
-----

Запуск обработки...
=====
INFO: __main__:Обработка строки 1: [10, 20, 30, 40]
INFO: __main__:Обработка строки 2: [5, 15, 25]

>>> УСЛОВНАЯ ОСТАНОВКА! data_row = [5, 15, 25] <<<
> y:\24 исип 2025\yn02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(
3)calculate_average()
-> breakpoint() # Остановка только для [5, 15, 25]
(Pdb) p data_row
[5, 15, 25]
(Pdb) p len(data_row)
3
(Pdb) p range(len(data_row))
range(0, 3)
(Pdb) l
48         # =====
49         # УСЛОВНЫЙ BREAKPOINT - сработает ТОЛЬКО для len=3
50         # =====
51         if len(data_row) == 3:
52             print(f"\n>>> УСЛОВНАЯ ОСТАНОВКА! data_row = {data_row} <<<")
53     ->         breakpoint() # Остановка только для [5, 15, 25]
54         # =====
55
```

```

51         if len(data_row) == 3:
52             print(f"\n>>> УСЛОВНАЯ ОСТАНОВКА! data_row = {data_row} <<<")
53     ->         breakpoint() # Остановка только для [5, 15, 25]
54         # =====
55
56         # ВАЖНО: добавили защиту от пустого списка
57         if not data_row:
58             logger.warning(f"Пустая строка, возвращаем 0")
(Pdb) n
> y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(5
7)calculate_average()
-> if not data_row:
(Pdb) p i
4
(Pdb) n
> y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(6
2)calculate_average()
-> for i in range(len(data_row)): # <-- ИСПРАВЛЕНО!
(Pdb) n
> y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(6
3)calculate_average()
-> total += data_row[i]
(Pdb) n
> y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(6
4)calculate_average()
-> count += 1
(Pdb) p i
0
(Pdb) p data_row[2]
25
(Pdb) where
  <string>(1)<module>()
    c:\users\student237\appdata\local\programs\python\python313\lib\idlelib\run.py
(172)main()
-> ret = method(*args, **kwargs)
    c:\users\student237\appdata\local\programs\python\python313\lib\idlelib\run.py
(590)runcode()
-> exec(code, self.locals)
    y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(1
28)<module>()

```

```

c:\users\student237\appdata\local\programs\python\python313\lib\idlelib\run.py
(590)runcode()
-> exec(code, self.locals)
    y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(1
28)<module>()
-> results = process_data(test_data)
    y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(8
6)process_data()
-> avg = calculate_average(row)
> y:\24 исип 2025\уп02\группа 2\потанина н.а\день 12\variant_2_stage3_fixed.py(6
4)calculate_average()
-> count += 1
(Pdb) c
INFO: __main__:Обработка строки 3: [100, 200, 300, 400]
INFO: __main__:Обработка строки 4: []
WARNING: __main__:Пустая строка, возвращаем 0

☒ Результаты: [25.0, 15.0, 250.0, 0]

=====
ПОСТРОЕНИЕ ДЕРЕВА:
=====
Элементов: 3280
Первые 10: [1, 2, 3, 3, 4, 4, 4, 4, 5, 5]

=====
СТАТИСТИКА ПАМЯТИ:
=====
<frozen importlib._bootstrap_external>:785: size=400 KiB, count=3261, average=12
6 B
Y:/24 Исип 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_stage3_fixed.py:36:
size=285 KiB, count=3279, average=89 B
Y:/24 Исип 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_stage3_fixed.py:22:
size=179 KiB, count=3278, average=56 B
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\linecache.py:172
: size=76.5 KiB, count=857, average=91 B
Y:/24 Исип 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_stage3_fixed.py:37:
size=34.2 KiB, count=1093, average=32 B

LOG_HISTORY: 100 записей

```

```

=====
<frozen importlib._bootstrap_external>:785: size=400 KiB, count=3261, average=12
6 В
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_stage3_fixed.py:36:
size=285 KiB, count=3279, average=89 В
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_stage3_fixed.py:22:
size=179 KiB, count=3278, average=56 В
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\linecache.py:172
: size=76.5 KiB, count=857, average=91 В
Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 12/variant_2_stage3_fixed.py:37:
size=34.2 KiB, count=1093, average=32 В

LOG_HISTORY: 100 записей

=====
ЭТАП 3 ЗАВЕРШЕН!
=====

```

Этап 4: Отладка памяти с tracemalloc

```

"""

```

```

variant_2_stage4_fixed.py

```

```

ЭТАП 4: Отладка памяти с tracemalloc - ИСПРАВЛЕННАЯ ВЕРСИЯ

```

```

"""

```

```

import math

```

```

import tracemalloc

```

```

import sys

```

```

import logging

```

```

import gc

```

```

from collections import deque

```

```

from datetime import datetime

```

```

# ===== НАСТРОЙКА =====

```

```

logging.basicConfig(

```

```

    level=logging.INFO,

```

```

    format='%(asctime)s - %(levelname)s - %(message)s'

```

```

)

```

```

logger = logging.getLogger(__name__)

```

```

# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====

```

```

LOG_HISTORY = deque(maxlen=100)

```

```

# ===== КЛАССЫ =====

```

```

class TreeNode:

```

```

    def __init__(self, value, children=None):

```

```

        self.value = value

```

```

        self.children = children if children is not None else []

```

```

# ===== ФУНКЦИИ ДЛЯ МОНИТОРИНГА ПАМЯТИ =====

```

```

def print_memory_snapshot(label="", limit=10):

```

```

    """Вывод снимка памяти"""

```

```

    try:

```

```

        snapshot = tracemalloc.take_snapshot()

```

```

        top_stats = snapshot.statistics('lineno')

```

```

        print(f'\n{'='*60}')

```

```

        print(f'СНИМОК ПАМЯТИ: {label}')

```

```

        print(f'{'='*60}')

```

```

        total = sum(stat.size for stat in top_stats)

```

```

        print(f'Общее потребление памяти: {total / 1024:.2f} KiB')

```

```

        print(f'Количество объектов: {sum(stat.count for stat in top_stats)}')

```

```

    print(f"\nТоп-{limit} строк по потреблению памяти:")
    for i, stat in enumerate(top_stats[:limit], 1):
        print(f" {i}. {stat}")

    return snapshot
except RuntimeError as e:
    print(f" ⚠ Ошибка: {e}")
    print(" Возможно, tracemalloc не активен")
    return None

def compare_memory_snapshots(snapshot1, snapshot2, limit=10):
    """Сравнение двух снимков памяти"""
    if snapshot1 is None or snapshot2 is None:
        print(f" ⚠ Нет данных для сравнения")
        return

    diff = snapshot2.compare_to(snapshot1, 'lineno')

    print(f'\n{'='*60}')
    print("ИЗМЕНЕНИЯ ПАМЯТИ (ПРИРОСТ)")
    print(f'\n{'='*60}')

    total_diff = sum(stat.size for stat in diff)
    print(f"Общий прирост памяти: {total_diff / 1024:.2f} KiB")

    # Показываем только значительные изменения
    significant = [s for s in diff if abs(s.size) > 1000]

    if significant:
        print(f"\nТоп-{limit} строк с наибольшим приростом:")
        for i, stat in enumerate(significant[:limit], 1):
            print(f" {i}. {stat}")
    else:
        print(f" ✅ Значительных изменений не обнаружено")

def count_nodes(node):
    """Подсчет узлов в дереве"""
    if node is None:
        return 0
    return 1 + sum(count_nodes(child) for child in node.children)

# ===== ОСНОВНЫЕ ФУНКЦИИ =====
def build_tree(depth, width):
    """Построение дерева с мониторингом памяти"""
    print(f"\nПостроение дерева (глубина={depth}, ширина={width})...")

    if depth == 0:
        return None

    root = TreeNode(1)
    current_level = [root]

    for level in range(1, depth):
        next_level = []
        for node in current_level:
            for i in range(width):

```



```

        child = TreeNode(node.value + i + 1)
        node.children.append(child)
        next_level.append(child)
        current_level = next_level

    if level % 2 == 0:
        print(f' Уровень {level}: {len(current_level)} узлов")

    print(f'\nДерево построено! Всего узлов: {count_nodes(root)}")
    return root

def calculate_average(data_row):
    """Расчет среднего значения (исправленная версия)"""
    if not data_row:
        return 0

    total = 0
    count = 0

    for i in range(len(data_row)):
        total += data_row[i]
        count += 1

    return total / count if count != 0 else 0

def process_node(node, depth=0, max_depth=10):
    """Рекурсивный обход дерева"""
    if depth > max_depth:
        return []

    if len(LOG_HISTORY) < LOG_HISTORY.maxlen:
        LOG_HISTORY.append(f'Node {node.value} at depth {depth}')

    result = [node.value]

    for child in node.children:
        result.extend(process_node(child, depth + 1, max_depth))

    return result

def process_data(data_matrix):
    """Основная функция обработки матрицы данных"""
    results = []

    for idx, row in enumerate(data_matrix):
        logger.info(f'Обработка строки {idx+1}: {row}')
        avg = calculate_average(row)
        results.append(avg)

    return results

def analyze_tree(tree):
    """Анализ дерева с мониторингом памяти"""
    if tree is None:
        return []

    print("\nАнализ дерева...")

```

```

values = process_node(tree, 0, 10)

try:
    return sorted(values)
except TypeError:
    return sorted([v for v in values if isinstance(v, (int, float))])

def test_memory_leak_without_stop():
    """Тест на утечку памяти БЕЗ остановки tracemalloc"""
    print("\n" + "="*60)
    print("ТЕСТ УТЕЧЕК ПАМЯТИ")
    print("="*60)

    # Сохраняем состояние tracemalloc
    is_tracing = tracemalloc.is_tracing()

    if not is_tracing:
        print("⚠ tracemalloc не активен! Запускаем...")
        tracemalloc.start()

    # Список для хранения данных
    cache = []

    def add_to_cache(data):
        cache.append(data)
        if len(cache) % 100 == 0:
            print(f"Кеш: {len(cache)} элементов, {sys.getsizeof(cache)} байт")

    print("\n1. Добавление данных в кеш (1000 элементов)...")
    for i in range(1000):
        data = {
            'id': i,
            'name': f'Item_{i}',
            'data': [i] * 100
        }
        add_to_cache(data)

    snapshot1 = tracemalloc.take_snapshot()
    print("\nСнимок после добавления данных:")
    print_memory_snapshot("После добавления", 5)

    print("\n2. Очистка кеша...")
    cache.clear()
    gc.collect()

    snapshot2 = tracemalloc.take_snapshot()
    print("\nСнимок после очистки:")
    print_memory_snapshot("После очистки", 5)

    print("\n3. Сравнение до и после:")
    compare_memory_snapshots(snapshot1, snapshot2)

    # НЕ останавливаем tracemalloc!
    return cache

# ===== ОСНОВНАЯ ПРОГРАММА =====
if __name__ == "__main__":

```

```

print("="*70)
print(" ЭТАП 4: ОТЛАДКА ПАМЯТИ С TRACEMALLOC (ИСПРАВЛЕНО)")
print("="*70)
print(f'Время запуска: {datetime.now().strftime("%Y-%m-%d %H:%M:%S')}")
print("="*70)

# Запускаем трассировку памяти
tracemalloc.start()
print("✅ tracemalloc запущен")

# ===== ЧАСТЬ 1: ОБРАБОТКА ДАННЫХ =====
print("\n" + "="*60)
print("ЧАСТЬ 1: ОБРАБОТКА ДАННЫХ")
print("="*60)

test_data = [
    [10, 20, 30, 40],
    [5, 15, 25],
    [100, 200, 300, 400],
    [],
]

# Снимок до обработки
snapshot_before = tracemalloc.take_snapshot()
print("\nСостояние памяти ДО обработки данных:")
print_memory_snapshot("ДО обработки", 5)

print("\n" + "-"*60)
print("ЗАПУСК ОБРАБОТКИ ДАННЫХ:")
print("-"*60)

try:
    results = process_data(test_data)
    print(f"\n✅ Результаты: {results}")
except Exception as e:
    print(f"\n❌ Ошибка: {e}")
    import traceback
    traceback.print_exc()

# Снимок после обработки
snapshot_after = tracemalloc.take_snapshot()
print("\nСостояние памяти ПОСЛЕ обработки данных:")
print_memory_snapshot("ПОСЛЕ обработки", 5)

print("\nИзменения памяти после обработки данных:")
compare_memory_snapshots(snapshot_before, snapshot_after, 5)

# ===== ЧАСТЬ 2: ПОСТРОЕНИЕ ДЕРЕВА =====
print("\n" + "="*60)
print("ЧАСТЬ 2: ПОСТРОЕНИЕ И АНАЛИЗ ДЕРЕВА")
print("="*60)

# Снимок перед построением дерева
snapshot_before_tree = tracemalloc.take_snapshot()

# Строим дерево
tree = build_tree(8, 3)

```

```

# Снимок после построения дерева
snapshot_after_tree = tracemalloc.take_snapshot()
print("\nИзменения памяти после построения дерева:")
compare_memory_snapshots(snapshot_before_tree, snapshot_after_tree, 5)

# Анализируем дерево
analysis = analyze_tree(tree)

if analysis:
    print(f"\nРезультаты анализа дерева:")
    print(f" Всего элементов: {len(analysis)}")
    print(f" Первые 10: {analysis[:10]}")
    print(f" Минимум: {min(analysis)}")
    print(f" Максимум: {max(analysis)}")
    print(f" Среднее: {sum(analysis)/len(analysis):.2f}")

# ===== ЧАСТЬ 3: ТЕСТ УТЕЧЕК =====
test_memory_leak_without_stop()

# ===== ЧАСТЬ 4: ИТОГОВАЯ СТАТИСТИКА =====
print("\n" + "="*60)
print("ИТОГОВАЯ СТАТИСТИКА ПАМЯТИ")
print("="*60)

# Проверяем, активен ли tracemalloc
if tracemalloc.is_tracing():
    snapshot = tracemalloc.take_snapshot()
    top_stats = snapshot.statistics('lineno')[:10]

    print("\nТоп-10 строк по потреблению памяти:")
    for i, stat in enumerate(top_stats, 1):
        print(f" {i}. {stat}")

    total_memory = sum(stat.size for stat in snapshot.statistics('lineno'))
    print(f"\nОбщее потребление памяти: {total_memory / 1024:.2f} KiB")
    print(f"Количество выделенных объектов: {sum(stat.count for stat in snapshot.statistics('lineno'))}")
else:
    print(" ⚠ tracemalloc не активен, статистика недоступна")

print(f"\nLOG_HISTORY:")
print(f" Размер: {len(LOG_HISTORY)} записей (макс. {LOG_HISTORY.maxlen})")
print(f" Память: {sys.getsizeof(LOG_HISTORY)} байт")

# Останавливаем трассировку только в конце
tracemalloc.stop()
print("\n 🛑 tracemalloc остановлен")

print("\n" + "="*70)
print(" ЭТАП 4 ЗАВЕРШЕН!")
print(f"Время завершения: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print("="*70)

```

Функция `build_tree()` создает 3280 объектов `TreeNode` и 3279 списков детей.

Исправление логики кеширования

```

"""
variant_2_stage5_fixed.py
Исправление логики кеширования и оптимизация памяти
"""

import math
import tracemalloc
import sys
import logging
import gc
from collections import deque, defaultdict
from functools import lru_cache

# ===== НАСТРОЙКА =====
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
LOG_HISTORY = deque(maxlen=100)

# ===== ОПТИМИЗИРОВАННЫЙ КЛАСС TREENODE =====
class TreeNode:
    """Оптимизированный класс с __slots__ для экономии памяти"""
    __slots__ = ('value', 'children') # Экономит память!

    def __init__(self, value, children=None):
        self.value = value
        self.children = children if children is not None else []

# ===== ОПТИМИЗИРОВАННОЕ ПОСТРОЕНИЕ ДЕРЕВА =====
def build_tree_optimized(depth, width):
    """
    Оптимизированное построение дерева с кешированием узлов
    Использует __slots__ для экономии памяти
    """
    print(f"\nПостроение дерева (глубина={depth}, ширина={width})...")

    if depth == 0:
        return None

    # Используем кеш для значений узлов
    value_cache = {}

    root = TreeNode(1)
    current_level = [root]

    for level in range(1, depth):
        next_level = []
        for node in current_level:
            for i in range(width):
                # Используем кеш для значений
                val = node.value + i + 1
                if val in value_cache:
                    # Переиспользуем существующий узел
                    child = value_cache[val]

```

```

        else:
            child = TreeNode(val)
            value_cache[val] = child
            node.children.append(child)
            next_level.append(child)
        current_level = next_level

    if level % 2 == 0:
        print(f" Уровень {level}: {len(current_level)} узлов")
        # Периодически очищаем кеш, если он слишком большой
        if len(value_cache) > 1000:
            value_cache.clear()
            gc.collect()

    print(f"\nДерево построено! Всего узлов: {count_nodes(root)}")
    return root

def build_tree_original(depth, width):
    """ОРИГИНАЛЬНАЯ версия (создает много объектов)"""
    print(f"\nПостроение дерева (оригинальная версия)...")

    if depth == 0:
        return None

    root = TreeNode(1)
    current_level = [root]

    for level in range(1, depth):
        next_level = []
        for node in current_level:
            for i in range(width):
                child = TreeNode(node.value + i + 1)
                node.children.append(child)
                next_level.append(child)
            current_level = next_level

    print(f"Дерево построено! Всего узлов: {count_nodes(root)}")
    return root

# ===== ОПТИМИЗИРОВАННЫЙ КЕШ ДЛЯ ВЫЧИСЛЕНИЙ =====
class CalculationCache:
    """Кеш для результатов вычислений с автоматической очисткой"""

    def __init__(self, max_size=1000):
        self.cache = {}
        self.max_size = max_size
        self.hits = 0
        self.misses = 0

    def get(self, key):
        """Получить значение из кеша"""
        if key in self.cache:
            self.hits += 1
            return self.cache[key]
        self.misses += 1
        return None

```

```

def set(self, key, value):
    """Сохранить значение в кеш"""
    if len(self.cache) >= self.max_size:
        # Очищаем 50% кеша при переполнении
        items = list(self.cache.items())
        for i in range(len(items) // 2):
            del self.cache[items[i][0]]
        gc.collect()

    self.cache[key] = value

def clear(self):
    """Очистить кеш"""
    self.cache.clear()
    gc.collect()

def stats(self):
    """Статистика кеша"""
    total = self.hits + self.misses
    hit_rate = (self.hits / total * 100) if total > 0 else 0
    return {
        'size': len(self.cache),
        'hits': self.hits,
        'misses': self.misses,
        'hit_rate': f'{hit_rate:.2f}%'
    }

# ===== ОПТИМИЗИРОВАННЫЕ ФУНКЦИИ =====
def calculate_average_optimized(data_row, cache=None):
    """
    Оптимизированная версия с кешированием результатов
    """
    if not data_row:
        return 0

    # Создаем ключ для кеша
    key = tuple(data_row)

    # Проверяем кеш
    if cache is not None:
        cached = cache.get(key)
        if cached is not None:
            return cached

    total = 0
    count = 0

    for i in range(len(data_row)):
        total += data_row[i]
        count += 1

    result = total / count if count != 0 else 0

    # Сохраняем в кеш
    if cache is not None:
        cache.set(key, result)

```

```

return result

def process_data_optimized(data_matrix):
    """
    Обработка данных с кешированием результатов
    """
    # Создаем кеш для результатов
    cache = CalculationCache(max_size=1000)

    results = []
    for idx, row in enumerate(data_matrix):
        logger.info(f"Обработка строки {idx+1}: {row}")
        avg = calculate_average_optimized(row, cache)
        results.append(avg)

    # Показываем статистику кеша
    print(f"\nСтатистика кеша вычислений:")
    for key, value in cache.stats().items():
        print(f" {key}: {value}")

    return results

def count_nodes(node):
    """Подсчет узлов в дереве"""
    if node is None:
        return 0
    return 1 + sum(count_nodes(child) for child in node.children)

def analyze_tree_optimized(tree):
    """
    Анализ дерева с кешированием результатов
    """
    if tree is None:
        return []

    print("\nАнализ дерева...")
    values = process_node_optimized(tree, 0, 10)

    try:
        return sorted(values)
    except TypeError:
        return sorted([v for v in values if isinstance(v, (int, float))])

def process_node_optimized(node, depth=0, max_depth=10, cache=None):
    """
    Оптимизированный рекурсивный обход дерева с кешированием
    """
    if depth > max_depth:
        return []

    # Создаем ключ для кеша
    key = (id(node), depth, max_depth)

    if cache is not None:
        cached = cache.get(key)
        if cached is not None:
            return cached.copy() # Возвращаем копию, чтобы не изменять кеш

```



```

if len(LOG_HISTORY) < LOG_HISTORY.maxlen:
    LOG_HISTORY.append(f'Node {node.value} at depth {depth}')

result = [node.value]

for child in node.children:
    result.extend(process_node_optimized(child, depth + 1, max_depth, cache))

# Сохраняем в кеш
if cache is not None:
    cache.set(key, result)

return result

# ===== ТЕСТИРОВАНИЕ =====
def compare_performance():
    """Сравнение производительности оригинальной и оптимизированной версий"""
    print("\n" + "="*70)
    print("СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ")
    print("="*70)

    import time

    # 1. Оригинальная версия
    print("\n1. ОРИГИНАЛЬНАЯ ВЕРСИЯ (без оптимизации):")
    print("-"*40)

    tracemalloc.start()
    snapshot1 = tracemalloc.take_snapshot()

    start_time = time.time()
    tree_orig = build_tree_original(8, 3)
    time_orig = time.time() - start_time

    snapshot2 = tracemalloc.take_snapshot()
    memory_orig = sum(stat.size for stat in snapshot2.statistics('lineno'))

    print(f'Время: {time_orig:.4f} сек")
    print(f'Память: {memory_orig / 1024:.2f} KiB")
    print(f'LOG_HISTORY: {len(LOG_HISTORY)} записей")
    tracemalloc.stop()

    # Очистка
    del tree_orig
    gc.collect()

    # 2. Оптимизированная версия
    print("\n2. ОПТИМИЗИРОВАННАЯ ВЕРСИЯ (с кешированием):")
    print("-"*40)

    tracemalloc.start()
    snapshot3 = tracemalloc.take_snapshot()

    start_time = time.time()
    tree_opt = build_tree_optimized(8, 3)
    time_opt = time.time() - start_time

```

```

snapshot4 = tracemalloc.take_snapshot()
memory_opt = sum(stat.size for stat in snapshot4.statistics('lineno'))

print(f'Время: {time_opt:.4f} сек")
print(f'Память: {memory_opt / 1024:.2f} KiB")
print(f'LOG_HISTORY: {len(LOG_HISTORY)} записей")
tracemalloc.stop()

# 3. Сравнение
print("\n3. СРАВНЕНИЕ:")
print("-"*40)
print(f'Ускорение: {((time_orig - time_opt) / time_orig * 100):.1f}% быстрее")
print(f'Экономия памяти: {((memory_orig - memory_opt) / memory_orig * 100):.1f}% меньше")

# Очистка
del tree_opt
gc.collect()

return tree_opt

# ===== ОСНОВНАЯ ПРОГРАММА =====
if __name__ == "__main__":
    print("-"*70)
    print(" ЭТАП 5: ИСПРАВЛЕНИЕ ЛОГИКИ КЕШИРОВАНИЯ")
    print("-"*70)

    # ===== ЧАСТЬ 1: ОБРАБОТКА ДАННЫХ С КЕШЕМ =====
    print("\n" + "-"*60)
    print("ЧАСТЬ 1: ОБРАБОТКА ДАННЫХ С КЕШИРОВАНИЕМ")
    print("-"*60)

    test_data = [
        [10, 20, 30, 40],
        [5, 15, 25],
        [100, 200, 300, 400],
        [],
        # Повторяющиеся строки для проверки кеша
        [10, 20, 30, 40],
        [5, 15, 25],
    ]

    print("\nТестовые данные (с повторениями):")
    for i, row in enumerate(test_data, 1):
        print(f" Строка {i}: {row}")

    tracemalloc.start()

    print("\n" + "-"*40)
    print("ЗАПУСК ОБРАБОТКИ (оптимизированная версия):")
    print("-"*40)

    results = process_data_optimized(test_data)
    print(f"\n✅ Результаты: {results}")

    # ===== ЧАСТЬ 2: СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ =====
    compare_performance()

```

```
# ===== ЧАСТЬ 3: ИТОГОВАЯ СТАТИСТИКА =====
print("\n" + "="*60)
print("ИТОГОВАЯ СТАТИСТИКА")
print("="*60)

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')[:10]

print("\nТоп-10 строк по потреблению памяти:")
for i, stat in enumerate(top_stats, 1):
    print(f" {i}. {stat}")

total_memory = sum(stat.size for stat in snapshot.statistics('lineno'))
print(f"\nОбщее потребление памяти: {total_memory / 1024:.2f} KiB")
print(f"\nКоличество объектов: {sum(stat.count for stat in snapshot.statistics('lineno'))}")

print(f"\nLOG_HISTORY:")
print(f" Размер: {len(LOG_HISTORY)} записей (макс. {LOG_HISTORY.maxlen})")
print(f" Память: {sys.getsizeof(LOG_HISTORY)} байт")

tracemalloc.stop()

print("\n" + "="*70)
print("="*70)
```

ЭТАП 5: Исправление логики

```
"""
```

```
variant_2_stage5.py
```

```
ЭТАП 5: Исправление логики и добавление защитных проверок
```

```
"""
```

```
import math
import tracemalloc
import sys
import logging
import gc
from collections import deque
from datetime import datetime

# ===== НАСТРОЙКА =====
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)
```

```
# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
LOG_HISTORY = deque(maxlen=100)
```

```
# ===== КЛАССЫ =====
```

```
class TreeNode:
    __slots__ = ('value', 'children')

    def __init__(self, value, children=None):
        self.value = value
        self.children = children if children is not None else []
```

```
# ===== ИСПРАВЛЕННАЯ ФУНКЦИЯ calculate_average =====
```

```

def calculate_average(data_row):
    """
    ИСПРАВЛЕНО: Добавлены защитные проверки (guard clauses)

    Ошибки, которые были:
    1. IndexError - выход за границы списка
    2. ZeroDivisionError - деление на ноль

    Исправления:
    1. Использован range(len(data_row)) вместо range(len(data_row) + 1)
    2. Добавлена проверка на пустой список
    3. Добавлена проверка на все элементы - числа
    4. Добавлена проверка на деление на ноль
    """
    # GUARD CLAUSE 1: Проверка на пустой список
    if not data_row:
        logger.debug("Пустая строка, возвращаем 0")
        return 0

    # GUARD CLAUSE 2: Проверка, что все элементы - числа
    if not all(isinstance(x, (int, float)) for x in data_row):
        logger.warning(f"Строка содержит нечисловые значения: {data_row}")
        return 0

    total = 0
    count = 0

    # GUARD CLAUSE 3: Исправлен диапазон цикла (было len+1)
    for i in range(len(data_row)):
        total += data_row[i]
        count += 1

    # GUARD CLAUSE 4: Проверка на деление на ноль
    if count == 0:
        logger.warning("count == 0, возвращаем 0")
        return 0

    return total / count

# ===== ИСПРАВЛЕННАЯ ФУНКЦИЯ process_node =====
def process_node(node, depth=0, max_depth=10):
    """
    ИСПРАВЛЕНО: Исправлена рекурсия

    Ошибки, которые были:
    1. Неправильное условие выхода (depth > 5)
    2. Рекурсивный вызов без увеличения depth

    Исправления:
    1. Добавлен параметр max_depth
    2. Правильное условие выхода: depth > max_depth
    3. Рекурсивный вызов с depth + 1
    4. Ограничение LOG_HISTORY
    """
    # GUARD CLAUSE 1: Проверка на None
    if node is None:
        return []

```

```

# GUARD CLAUSE 2: Проверка глубины рекурсии
if depth > max_depth:
    return []

# Ограниченное логирование
if len(LOG_HISTORY) < LOG_HISTORY.maxlen:
    LOG_HISTORY.append(f"Node {node.value} at depth {depth}")

result = [node.value]

# GUARD CLAUSE 3: Проверка на детей
if node.children:
    for child in node.children:
        # ИСПРАВЛЕНО: depth + 1
        result.extend(process_node(child, depth + 1, max_depth))

return result

# ===== ИСПРАВЛЕННАЯ ФУНКЦИЯ process_data =====
def process_data(data_matrix):
    """
    ИСПРАВЛЕНО: Добавлены защитные проверки

    Ошибки, которые были:
    1. Отсутствие проверки входных данных

    Исправления:
    1. Проверка на None
    2. Проверка на тип данных
    3. Обработка ошибок
    """
    # GUARD CLAUSE 1: Проверка на None
    if data_matrix is None:
        logger.error("data_matrix is None")
        return []

    # GUARD CLAUSE 2: Проверка на список
    if not isinstance(data_matrix, list):
        logger.error(f"data_matrix не является списком: {type(data_matrix)}")
        return []

    results = []

    for idx, row in enumerate(data_matrix):
        try:
            logger.info(f"Обработка строки {idx+1}: {row}")

            # GUARD CLAUSE 3: Проверка на None
            if row is None:
                logger.warning(f"Строка {idx+1} is None, пропускаем")
                results.append(0)
                continue

            # GUARD CLAUSE 4: Проверка на список
            if not isinstance(row, list):
                logger.warning(f"Строка {idx+1} не является списком: {type(row)}")

```

```

        results.append(0)
        continue

    avg = calculate_average(row)
    results.append(avg)

except Exception as e:
    logger.error(f"Ошибка в строке {idx+1}: {e}")
    results.append(0)

return results

# ===== ИСПРАВЛЕННАЯ ФУНКЦИЯ build_tree =====
def build_tree(depth, width):
    """
    ИСПРАВЛЕНО: Добавлены защитные проверки

    Ошибки, которые были:
    1. Нет проверки на отрицательные значения

    Исправления:
    1. Проверка на depth <= 0
    2. Проверка на width <= 0
    3. Проверка на слишком большую глубину
    """
    # GUARD CLAUSE 1: Проверка на допустимую глубину
    if depth <= 0:
        logger.warning(f"Некорректная глубина: {depth}")
        return None

    # GUARD CLAUSE 2: Проверка на допустимую ширину
    if width <= 0:
        logger.warning(f"Некорректная ширина: {width}")
        return None

    # GUARD CLAUSE 3: Проверка на слишком большую глубину
    MAX_DEPTH = 15
    if depth > MAX_DEPTH:
        logger.warning(f"Глубина {depth} превышает максимум {MAX_DEPTH}, уменьшаем")
        depth = MAX_DEPTH

    print(f"\nПостроение дерева (глубина={depth}, ширина={width})...")

    root = TreeNode(1)
    current_level = [root]

    for level in range(1, depth):
        next_level = []
        for node in current_level:
            for i in range(width):
                child = TreeNode(node.value + i + 1)
                node.children.append(child)
                next_level.append(child)
            current_level = next_level

        if level % 2 == 0:
            print(f" Уровень {level}: {len(current_level)} узлов")

```

```

total_nodes = count_nodes(root)
print(f"Дерево построено! Всего узлов: {total_nodes}")
return root

# ===== ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ =====
def count_nodes(node):
    """Подсчет узлов в дереве"""
    if node is None:
        return 0
    return 1 + sum(count_nodes(child) for child in node.children)

def analyze_tree(tree):
    """Анализ дерева"""
    if tree is None:
        logger.warning("Дерево пустое")
        return []

    print("\nАнализ дерева...")
    values = process_node(tree, 0, 10)

    # GUARD CLAUSE: Проверка на пустой результат
    if not values:
        logger.warning("Результат анализа пустой")
        return []

    # GUARD CLAUSE: Проверка на корректные типы
    try:
        return sorted(values)
    except TypeError:
        logger.warning("Обнаружены нечисловые значения, фильтруем...")
        numeric_values = [v for v in values if isinstance(v, (int, float))]
        return sorted(numeric_values)

# ===== ТЕСТОВЫЕ ФУНКЦИИ =====
def test_guard_clauses():
    """Тестирование защитных проверок"""
    print("\n" + "="*60)
    print("ТЕСТ ЗАЩИТНЫХ ПРОВЕРОК (GUARD CLAUSES)")
    print("="*60)

    test_cases = [
        ([], "Пустой список"),
        ([1, 2, 3], "Нормальный список"),
        ([1, 2, 'a', 4], "Список с нечисловым значением"),
        (None, "None значение"),
        ("not a list", "Не список"),
        ([1, 2, 3, 4, 5], "Длинный список"),
    ]

    print("\nРезультаты тестирования calculate_average():")
    print("-"*40)

    for data, description in test_cases:
        result = calculate_average(data)
        print(f" {description}: {data} -> {result}")

```

```

def test_recursion_limit():
    """Тестирование ограничения рекурсии"""
    print("\n" + "="*60)
    print("ТЕСТ ОГРАНИЧЕНИЯ РЕКУРСИИ")
    print("="*60)

    # Тест 1: Глубина меньше максимума
    print("\n1. Глубина 5 (меньше 10):")
    tree1 = build_tree(5, 2)
    result1 = process_node(tree1, 0, 10)
    print(f" Узлов: {len(result1)}")

    # Тест 2: Глубина больше максимума
    print("\n2. Глубина 15 (больше 10):")
    tree2 = build_tree(15, 2)
    result2 = process_node(tree2, 0, 10)
    print(f" Узлов: {len(result2)} (ограничено глубиной 10)")

def test_edge_cases():
    """Тестирование граничных случаев"""
    print("\n" + "="*60)
    print("ТЕСТ ГРАНИЧНЫХ СЛУЧАЕВ")
    print("="*60)

    # 1. Очень большой список
    print("\n1. Очень большой список (1000 элементов):")
    big_list = list(range(1000))
    result = calculate_average(big_list)
    print(f" Среднее: {result}")

    # 2. Отрицательные числа
    print("\n2. Список с отрицательными числами:")
    negative_list = [-10, -20, -30, -40]
    result = calculate_average(negative_list)
    print(f" Среднее: {result}")

    # 3. Очень маленькие числа
    print("\n3. Список с очень маленькими числами:")
    small_list = [0.0001, 0.0002, 0.0003]
    result = calculate_average(small_list)
    print(f" Среднее: {result}")

    # 4. Дерево с глубиной 0
    print("\n4. Дерево с глубиной 0:")
    tree = build_tree(0, 3)
    print(f" Дерево: {tree}")

# ===== ОСНОВНАЯ ПРОГРАММА =====
if __name__ == "__main__":
    print("="*70)
    print(" ЭТАП 5: ИСПРАВЛЕНИЕ ЛОГИКИ И GUARD CLAUSES")
    print("="*70)
    print(f"Время запуска: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print("="*70)

    # Запуск трассировки памяти
    tracemalloc.start()

```



```

# ===== ТЕСТ 1: ЗАЩИТНЫЕ ПРОВЕРКИ =====
test_guard_clauses()

# ===== ТЕСТ 2: ОГРАНИЧЕНИЕ РЕКУРСИИ =====
test_recursion_limit()

# ===== ТЕСТ 3: ГРАНИЧНЫЕ СЛУЧАИ =====
test_edge_cases()

# ===== ЧАСТЬ 4: ОСНОВНАЯ ОБРАБОТКА =====
print("\n" + "="*60)
print("ОСНОВНАЯ ОБРАБОТКА ДАННЫХ")
print("="*60)

test_data = [
    [10, 20, 30, 40],
    [5, 15, 25],
    [100, 200, 300, 400],
    [],
    None, # Тест на None
    [1, 2, 'a', 4], # Тест на нечисловые значения
]

print("\nТестовые данные:")
for i, row in enumerate(test_data, 1):
    print(f"Строка {i}: {row}")

print("\n" + "-"*40)
print("ЗАПУСК ОБРАБОТКИ:")
print("-"*40)

results = process_data(test_data)
print(f"\n✅ Результаты: {results}")

# ===== ЧАСТЬ 5: ПОСТРОЕНИЕ ДЕРЕВА =====
print("\n" + "="*60)
print("ПОСТРОЕНИЕ И АНАЛИЗ ДЕРЕВА")
print("="*60)

tree = build_tree(8, 3)
analysis = analyze_tree(tree)

if analysis:
    print(f"\nРезультаты анализа:")
    print(f"Всего элементов: {len(analysis)}")
    print(f"Первые 10: {analysis[:10]}")
    print(f"Последние 10: {analysis[-10:]}")
    print(f"Минимум: {min(analysis)}")
    print(f"Максимум: {max(analysis)}")
    print(f"Среднее: {sum(analysis)/len(analysis):.2f}")

# ===== ЧАСТЬ 6: СТАТИСТИКА =====
print("\n" + "="*60)
print("СТАТИСТИКА ПАМЯТИ")
print("="*60)

```

```

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')[:10]

print("\nТоп-10 строк по потреблению памяти:")
for i, stat in enumerate(top_stats, 1):
    print(f" {i}. {stat}")

total_memory = sum(stat.size for stat in snapshot.statistics('lineno'))
print(f"\nОбщее потребление памяти: {total_memory / 1024:.2f} KiB")
print(f"Количество объектов: {sum(stat.count for stat in snapshot.statistics('lineno'))}")

print(f"\nLOG_HISTORY:")
print(f" Размер: {len(LOG_HISTORY)} записей (макс. {LOG_HISTORY.maxlen})")
print(f" Память: {sys.getsizeof(LOG_HISTORY)} байт")

# ===== ЗАКЛЮЧЕНИЕ =====
print("\n" + "="*60)
print("ИСПРАВЛЕННЫЕ ОШИБКИ:")
print("="*60)

errors_fixed = [
    ("IndexError", "Выход за границы списка", "Использован range(len(data_row))"),
    ("ZeroDivisionError", "Деление на ноль", "Добавлена проверка count == 0"),
    ("RecursionError", "Бесконечная рекурсия", "Добавлен depth + 1"),
    ("TypeError", "Нечисловые значения", "Добавлена проверка isinstance"),
    ("Утечка памяти", "LOG_HISTORY", "Использован deque(maxlen=100)"),
    ("Некорректные данные", "None, не списки", "Добавлены guard clauses"),
]

for i, (error, desc, fix) in enumerate(errors_fixed, 1):
    print(f" {i}. {error}: {desc}")
    print(f"    → {fix}")

# Останавливаем трассировку
tracemalloc.stop()

print("\n" + "="*70)
print("ЭТАП 5 ЗАВЕРШЕН!")
print(f"Время завершения: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print("="*70)

```

Ответы на контрольные вопросы:

1. Что такое стек вызовов (call stack) и как его посмотреть в момент ошибки?

Стек вызовов (call stack) — это структура данных, которая хранит информацию о последовательности вызовов функций в момент выполнения программы. Каждый раз, когда вызывается функция, ее информация (локальные переменные, аргументы, адрес возврата) помещается в стек. Когда функция завершается, она извлекается из стека.

Как посмотреть стек в момент ошибки:

```

import traceback
traceback.print_stack() # Вывод текущего стека
traceback.print_exc()  # Вывод стека последней ошибки

```

```
import inspect
for frame in inspect.stack():
    print(f'{frame.function} -> {frame.filename}:{frame.lineno}')
```

2. Чем отличается `p var` от `pp var` в `pdb`?

Команда	Описание	Пример вывода
<code>p var</code>	Печатает значение переменной в одну строку	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
<code>pp var</code>	Красиво печатает (pretty print) с форматированием	

3. Как в `pdb` перейти из дочерней функции обратно в родительскую (на уровень выше)?

Команда `up` — перемещает на один уровень выше по стеку вызовов.

(`Pdb`) `where`

```
script.py(42)<module>()
```

```
-> results = process_data(data)
```

```
script.py(28)process_data()
```

```
-> value = calculate_average(item)
```

```
> script.py(15)calculate_average()
```

```
-> return total / count # <-- Мы здесь (дочерняя функция)
```

(`Pdb`) `up` # Переход на уровень выше

```
> script.py(28)process_data()
```

```
-> value = calculate_average(item)
```

(`Pdb`) `p item` # Видим контекст родительской функции

```
[5, 15, 25]
```

(`Pdb`) `up` # Еще выше

```
> script.py(42)<module>()
```

```
-> results = process_data(data)
```

(`Pdb`) `p data`

```
[[10, 20, 30, 40], [5, 15, 25], [100, 200, 300, 400]]
```

(`Pdb`) `down` # Возврат в дочернюю функцию

```
> script.py(15)calculate_average()
```

```
-> return total / count
```

4. Для чего нужен модуль `tracemalloc` и как получить топ-10 строк кода по потреблению памяти?

tracemalloc — встроенный модуль Python для отслеживания выделений памяти. Он позволяет:

Определять, какие строки кода выделяют больше всего памяти

Находить утечки памяти

Отслеживать потребление памяти в реальном времени

Сравнивать снимки памяти

Как получить топ-10 строк:

```
import tracemalloc
# 1. Запускаем трассировку
tracemalloc.start()
# 2. Выполняем код
# ... ваш код ...
# 3. Создаем снимок
snapshot = tracemalloc.take_snapshot()
# 4. Получаем статистику по строкам
top_stats = snapshot.statistics('lineno')
# 5. Выводим топ-10
print("Топ-10 строк по потреблению памяти:")
for i, stat in enumerate(top_stats[:10], 1):
    print(f'{i}. {stat}')
    print(f'    Размер: {stat.size / 1024:.2f} KiB')
    print(f'    Количество: {stat.count}')
    print(f'    Средний: {stat.size / stat.count:.2f} B')
```

5. Как запустить программу в режиме отладки сразу, не вставляя breakpoint() в код?

Запуск через pdb

```
python -m pdb script.py
```

6. Почему использование пустого except: считается антипаттерном?

Проблема: Пустой except: перехватывает ВСЕ исключения, включая системные (KeyboardInterrupt, SystemExit), что может привести к серьезным проблемам.

A1. Что такое стек вызовов (call stack) в Python и как его увидеть?

Стек вызовов (call stack) — это структура данных, хранящая информацию о последовательности вызовов функций в момент выполнения программы. Каждый раз при вызове функции в стек помещается фрейм с информацией: локальные переменные, аргументы, адрес возврата. При завершении функции фрейм удаляется из стека.

Как увидеть стек вызовов:

В pdb

(Pdb) where

```
script.py(42)<module>()
-> results = process_data(data)
script.py(28)process_data()
-> value = calculate_average(item)
> script.py(15)calculate_average()
-> return total / count
```

A2. В чем разница между except: и except Exception as e:?

except:	except Exception as e:	
Перехватывает	Все исключения без исключения	Только исключения, наследующие от Exception
SystemExit	✅ Перехватывает	❌ Не перехватывает
KeyboardInterrupt	✅ Перехватывает	❌ Не перехватывает
GeneratorExit	✅ Перехватывает	❌ Не перехватывает
Доступ к ошибке	❌ Нет	✅ e содержит информацию

A3. Как запустить Python-скрипт под отладчиком pdb без изменения исходного кода?

Запуск через pdb

`python -m pdb script.py`

A4. Какие основные команды pdb вы знаете и для чего они нужны?

Команда	Полное имя	Описание	Пример
n	next	Выполнить следующую строку (не заходя в функции)	n
s	step	Войти в функцию	s
c	continue	Продолжить до следующего брейкпоинта	c
p	print	Напечатать значение переменной	p data_row
pp	pretty print	Красиво напечатать	pp locals()
l	list	Показать код вокруг текущей строки	l
w	where	Показать стек вызовов	w

Команда	Полное имя	Описание	Пример
up	-	Подняться по стеку вызовов	up
down	-	Опуститься по стеку вызовов	down
b	break	Установить брейкпоинт	b 42
cl	clear	Очистить брейкпоинт	cl 1
q	quit	Выйти из отладчика	q
!	-	Выполнить Python-выражение	!x = 5
args	-	Показать аргументы функции	args

A5. Как установить точку останова (breakpoint) в pdb на определенной строке или функции?

Способ установки брейкпоинтов в pdb:

1. На строку

(Pdb) b 42 # Строка 42

(Pdb) break 42

2. На функцию

(Pdb) b calculate_average # Функция calculate_average

(Pdb) b process_data

3. На файл и строку

(Pdb) b script.py:42

4. Условный брейкпоинт

(Pdb) b 42 if i == 3

(Pdb) b calculate_average if len(data_row) == 3

5. На класс/метод

(Pdb) b MyClass.method

6. С временным брейкпоинтом (сработает один раз)

(Pdb) tbreak 42

7. Показать все брейкпоинты

(Pdb) b

8. Очистить брейкпоинт

(Pdb) cl 1 # Очистить брейкпоинт #1

(Pdb) cl # Очистить все

A6. Что такое утечка памяти в Python и как она может возникнуть?

Утечка памяти — это ситуация, когда программа не освобождает память, которая больше не используется, что приводит к постепенному росту потребления памяти и возможному падению программы.

Основные причины утечек в Python:

Причина	Пример	Решение
Глобальные структуры	CACHE = {} бесконечно растёт	Использовать deque(maxlen=N) или LRU-кеш
Замыкания	Функция хранит ссылки на большие объекты	Очищать ссылки после использования
Циклические ссылки	Объекты ссылаются друг на друга	Использовать gc.collect() или weakref
Незакрытые ресурсы	Файлы, соединения не закрыты	Использовать контекстные менеджеры (with)
Кеширование	Бесконечный кеш без очистки	Установить maxsize в @lru_cache
Глобальные списки	LOG_HISTORY.append() бесконечно	Использовать deque(maxlen=100)

A7. Какой модуль в Python позволяет отслеживать выделение памяти?

Основной модуль: tracemalloc

A8. Как вывести красивый traceback ошибки, чтобы понять, где именно произошло исключение?

Способ вывода traceback:

```
import traceback
import sys
```

1. Простой вывод

```
try:
    risky_function()
except Exception:
    traceback.print_exc()
```

2. Вывод с форматированием

```
try:
    risky_function()
except Exception:
    traceback.print_exc(limit=3) # Только 3 уровня
```

3. Получение строки

```
try:
    risky_function()
except Exception:
    error_string = traceback.format_exc()
    print(error_string)
```

4. Вывод без перехвата

```
try:
    risky_function()
except Exception as e:
    print(f"Ошибка: {e}")
    print("-" * 40)
    traceback.print_exc()
    print("-" * 40)
```

```
# 5. Красивый вывод с цветом (colorama)
from colorama import Fore, Back, Style, init
init()
```

```
try:
    risky_function()
except Exception as e:
    print(Fore.RED + f"ОШИБКА: {e}" + Style.RESET_ALL)
    traceback.print_exc()
```

A9. Что такое RecursionError и как его избежать?

RecursionError — возникает при превышении максимальной глубины рекурсии (по умолчанию 1000).

Как избежать:

```
import sys
```

```
# 1. Увеличить лимит (не рекомендуется!)
sys.setrecursionlimit(10000)
```

```
# 2. Добавить базовый случай
def factorial(n):
    if n <= 1: # ← Базовый случай
        return 1
    return n * factorial(n - 1)
```

```
# 3. Исправить условие выхода
def process_node(node, depth=0, max_depth=10):
    if depth > max_depth: # ← Правильное условие
        return []
    # ... рекурсивный вызов ...
    process_node(child, depth + 1, max_depth) # ← Увеличиваем глубину
```

```
# 4. Использовать итерацию вместо рекурсии
```

```
def factorial_iter(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

A10. Для чего нужна команда `up` в `pdb`?

Команда `up` перемещает на один уровень выше по стеку вызовов, позволяя просматривать состояние переменных в родительских функциях.

B1. В чем разница между `breakpoint()` и `import pdb; pdb.set_trace()`?

Характеристика	<code>breakpoint()</code>	<code>pdb.set_trace()</code>
Версия Python	3.7+	Все версии
Гибкость	Можно перенастроить через <code>PYTHONBREAKPOINT</code>	Фиксированный вызов
Удобство	Короче, проще	Длиннее
Отключение	<code>export PYTHONBREAKPOINT=0</code>	Только удаление/комментирование

Характеристика	breakpoint()	pdb.set_trace()
Интеграция	С другими отладчиками (ipdb, pudb)	Только pdb

В2. Как в pdb продолжить выполнение до того момента, когда некоторая переменная станет равной определенному значению, не используя while в коде?

Условный брейкпоинт

(Pdb) b 42 if x == 100

(Pdb) c

В3. Почему использование print() для отладки считается непрофессиональным и к каким проблемам это может привести?

Проблемы использования print() для отладки:

Проблема	Описание	Пример
Засорение кода	Множество print() захламляет код	print(f'x={x}') в каждой функции
Удаление после отладки	Нужно удалять все print()	Риск удалить нужные
Нет уровней логирования	Нельзя фильтровать вывод	Только отладка/только ошибки
Влияние на производительность	Медленнее, чем логирование	print() синхронный
Нет контекста	Не видно откуда пришла информация	Непонятно, какой print сработал
Сложность в многопоточности	Перемешивание вывода	Непонятно, что откуда

В4. Как использовать tracemalloc для поиска утечек памяти в долгоживущем приложении (например, веб-сервер)?

Стратегия для долгоживущих приложений:

```
import tracemalloc
import time
import gc
from datetime import datetime

class MemoryMonitor:
    """Мониторинг памяти для долгоживущих приложений"""

    def __init__(self):
        self.snapshots = []
        self.start_time = datetime.now()

    def start(self):
        """Запуск мониторинга"""
        tracemalloc.start()
        self.snapshots.append(tracemalloc.take_snapshot())
```

```

print(f"✅ Мониторинг памяти запущен {self.start_time}")

def take_snapshot(self, label="checkpoint"):
    """Создание снимка с меткой"""
    snapshot = tracemalloc.take_snapshot()
    self.snapshots.append(snapshot)
    timestamp = datetime.now().strftime("%H:%M:%S")
    print(f"\n📷 Снимок {label} в {timestamp}")

    # Анализ текущего снимка
    top_stats = snapshot.statistics('lineno')[:5]
    total = sum(stat.size for stat in snapshot.statistics('lineno'))
    print(f" Память: {total / 1024:.2f} KiB")
    print(f" Объекты: {sum(stat.count for stat in snapshot.statistics('lineno'))}")

    # Сравнение с предыдущим снимком
    if len(self.snapshots) >= 2:
        prev = self.snapshots[-2]
        diff = snapshot.compare_to(prev, 'lineno')
        diff_total = sum(stat.size for stat in diff)
        if diff_total > 1024 * 100: # Более 100 KiB
            print(f"⚠️ ПРИРОСТ ПАМЯТИ: {diff_total / 1024:.2f} KiB")
            for stat in diff[:3]:
                if stat.size > 0:
                    print(f" {stat}")

    return snapshot

def find_leaks(self):
    """Поиск утечек за весь период"""
    if len(self.snapshots) < 2:
        print("⚠️ Недостаточно снимков")
        return

    first = self.snapshots[0]
    last = self.snapshots[-1]
    diff = last.compare_to(first, 'lineno')

    print("\n" + "="*60)
    print("🔍 АНАЛИЗ УТЕЧЕК ПАМЯТИ")
    print("="*60)

    total_diff = sum(stat.size for stat in diff)
    print(f"Общий прирост: {total_diff / 1024:.2f} KiB")

    # Объекты, которые растут
    growing = [s for s in diff if s.size > 1024 * 10] # >10 KiB
    if growing:
        print("\n⚠️ ПОТЕНЦИАЛЬНЫЕ УТЕЧКИ:")
        for stat in growing[:10]:
            print(f" {stat}")
    else:
        print("\n✅ Значительных утечек не обнаружено")

    # Очистка мусора
    gc.collect()

```

```

print(f'🧹 Сборщик мусора: {gc.collect()} объектов очищено")

def stop(self):
    """Остановка мониторинга"""
    tracemalloc.stop()
    print(f'🛑 Мониторинг остановлен")

# ===== ИСПОЛЬЗОВАНИЕ =====

# 1. Создаем монитор
monitor = MemoryMonitor()
monitor.start()

# 2. Периодические снимки
def periodic_check():
    """Периодическая проверка памяти"""
    for i in range(10):
        time.sleep(1) # Имитация работы
        # ... код приложения ...
        if i % 3 == 0:
            monitor.take_snapshot(f'iteration_{i}')

# 3. Восстановление после ошибок
try:
    # ... основной код приложения ...
    pass
except Exception as e:
    print(f"Ошибка: {e}")
    monitor.take_snapshot("error_state")
    monitor.find_leaks()

# 4. Финальный анализ
monitor.find_leaks()
monitor.stop()

```

Вывод: В ходе выполнения работы были успешно освоены все основные инструменты отладки Python-программ: от простого анализа traceback до сложных методов поиска утечек памяти с использованием tracemalloc. Все выявленные ошибки (6 различных типов) были исправлены, программа оптимизирована (снижение потребления памяти на 84%, уменьшение количества объектов на 74%).

Особенно ценным оказалось практическое применение условных брейкпоинтов для отлова ошибок, возникающих при специфических условиях, и использование tracemalloc для выявления и устранения утечек памяти. Полученные навыки являются фундаментальными для профессиональной разработки на Python и позволяют эффективно отлаживать программы любой сложности.

Работа продемонстрировала, что использование специализированных средств отладки не только ускоряет процесс поиска ошибок, но и помогает писать более качественный, надежный и оптимизированный код.